

Sparse Matrix-Vector Multiplication (SpMV) Investigation

Titien MARIE
Erasmus Student
University of Trento, Italy
titienjacques.marie@studenti.unitn.it

I. INTRODUCTION

The Sparse Matrix-Vector Multiplication (SpMV) operation is a fundamental kernel in many scientific computing applications. This report evaluates the performance of five SpMV storage formats across a dataset of 10 matrices from the SuiteSparse collection.

II. METHODOLOGY

In this SpMV investigation, we first implemented CPU solutions such as COO, CSR, ELL, and SELL in a "naive" way in order to compare the GPU ones and also to copy the different arrays necessary for each method. In this part, we provide the pseudo-codes for each method for the two types of architectures (*see the subsections*). To have a reference in terms of GPU computation time and result references, we used the *cuSPARSE* implementation via the library provided by NVIDIA on their GitHub [2]. To cover our work, we took inspiration from the survey written by Gao, J. *et al.* [3], which helped us to have a global view of the state-of-the-art and the different optimizations possible. For the different results we will provide in this paper, we used an NVIDIA Ampere GPU (A30). You can find some specifications in this table:

TABLE I
NVIDIA A30 SPECIFICATIONS (*Via Cuda Device Properties*)

Memory clock rate	1215 Mhz
Memory Bus Width	3072 Mhz
Peak Memory bandwidth	933.120 GB/s
Multiprocessors	56
Max number of threads/multiprocessor	2048
Max number of threads/block	1024
Max dimension size of thread block (x,y,z)	(1024, 1024, 64)
Max dimension size of a grid size	(2147483647, 65535, 65535)
Total amount of shared memory per block	49152 bytes

As it was evoked above (I), we are trying to compare different implementations of SpMV with 2 types of dense vector x of size m , where m is the number of columns of the matrix $A \in \mathbb{R}^{n \times m}$: one with only "ones" in order to sum the elements of each row of the input matrix in each corresponding row of x for simpler verifications, and the other one is generated "on the fly" with random float values generated by the library provided by the C language (`random.h`) with a seed written by the user during the execution as a parameter. All along this investigation, we used *float32* type for the matrix and the dense vector.

For each method, we first run the CPU solution and then

we copy the arrays for the GPU with functions provided by NVIDIA [1] such as `cudaMemcpy`, directly in the global memory used specifically by the device. Each execution starts with a *warmup*, here we chose 2, then it iterates 20 times the SpMV computation by storing each measured time with CUDA events [1] in ms (*milliseconds*) and a *timeofday*, which has to be converted in milliseconds, in an array of timers (*float32*). Then, the arithmetic mean is computed on this timers' array.

Finally, each GPU SpMV solution (*vector y*) is compared to the CPU solution and the one provided by the *cuSPARSE* library with a relative error $e_{i,relative} < \epsilon < 10^{-4}$ for *float32*. In case an error is encountered, it is reported in the error stream (*stderr*) with a string indicating which kernel failed, which line, and the 2 values compared. In this other case, we specify it as "OK". As a reminder, we consider the relative error for each line index i of the vector y as follows: $e_{i,relative} = \frac{|v_{i,computed} - v_{i,real}|}{v_{i,real}}$. Here, $v_{i,real}$ is the one computed with the CPU, and $v_{i,computed}$ is the one computed with the GPU (*our kernels or cuSPARSE*).

We will see in the next sub-sections that we optimized the *naive* CSR with warps and shared memory. We consider SELL/ELL SpMV format as experimental formats due to the version of CUDA we used (11.8.0), which does not provide a *cuSPARSE* solution (*only for CUDA versions above 12.2.0 [2]*).

A. COO

In this first SpMV (Sparse-matrix Multiplication Vector) we consider a basic solution for CPU which translates each nz (*non-zero*) encountered in the file read line by line until the 'EOF' (*End Of File*) is reached by basic stream reading in C that consists on the COO format (*COOrdinates matrix format*) as 1 value for each 3 arrays of size nnz (*he Number of Non-Zeros*), respectively :

- Values array (*floats32*) : the value of the nz
- Columns array (*int*) : the column of the nz
- Lines array (*int*) : the line of the nz

For the sequential implementation of this SpMV it is intuitive, the pseudo-code can be written this way :

Algorithm 1 Sequential (via CPU) SpMV in COO format

Require: $y, rows, cols, vals, x, nnz$

```
1: for  $i = 0$  to  $nnz - 1$  do  
2:    $y[rows[i]] \leftarrow y[rows[i]] + vals[i] \times x[cols[i]]$   
3: end for
```

In this pseudo-code x is the vector that multiplies the matrix A on which we want to apply the SpMV with COO format, y the result vector and finally $rows, cols$ and $vals$ are the array corresponding to the format we described above (II-A). As we access to 3 vectors of nnz size and we travel through a loop with $i \in [0, nnz - 1]$ we can assume that the algorithmic complexity is approximatively $O(3nnz)$. The CPU by essence is crossing the loop sequentially, this method is naive and optimized. The main goal here is to compare with the GPU solution that can parallelize the computation.

In CUDA we have to specify for each kernel the *threadsPerBlock* and *blockSize*. In this solution we consider this $numBlocks = (nnz + threadsPerBlock - 1) / threadsPerBlock$ because we are working on nnz so the grid size has to be adapted to this unit. For the second parameter we consider that it is directly linked to the *device property* we use with a simple formula in order to allocate threads that does not cause overlap or under-computation :

Algorithm 2 BlockSize computation via device properties

```
1:  $cudaGetDeviceProperties(\&devProp, 0)$   
2:  $maxThreadsPerBlock \leftarrow devProp.maxThreadsPerBlock$   
3:  $threadsPerBlock \leftarrow \min(1024, maxThreadsPerBlock)$ 
```

We pass the *devProp (int)* with its address due to the fact that the function *cudaGetDeviceProperties* provided by CUDA API [1] directly change it inside via the specific device we use with its specific identifier (*id_device*), here for more clarity we consider only the device 0 but in reality we are going through the loop via the *deviceCount* variable and we are verifying for each one.

To investigate the efficiency of the GPU compared to CPU we provided this naive solution which uses the global memory (*_global*) :

Algorithm 3 SpMV in COO format (GPU kernel)

Require: $y, rows, cols, vals, x, nnz$

```
1:  $idx \leftarrow threadIdx.x + blockIdx.x \times blockDim.x$   
2: for  $number \leftarrow idx$  to  $nnz - 1$  step  $blockDim.x \times gridDim.x$  do  
3:    $r \leftarrow rows\_array[number]$   
4:    $c \leftarrow cols\_array[number]$   
5:    $v \leftarrow vals\_array[number]$   
6:    $atomicAdd(res[r], v \times vect[c])$   
7: end for
```

Note: atomicAdd is required due to concurrent updates on the same output vector entries.

threadIdx.x : position of the thread in the block

blockIdx.x : position of the block in the grid

blockDim.x : number of threads per block

The operations in this case are parallelized due to the GPU properties compared to CPU : in other words, each thread is responsible for processing a subset of non-zero elements using a grid-stride loop then it computes the product between

a matrix coefficient and the corresponding element of the input vector, and accumulates the result into the output vector.

Due to concurrent updates on the same output entries (*it is called : race condition*) atomic operations are required to ensure correctness. In other terms it happens because different non-zero elements may belong to the same row.

With this type of loop we see that it is not optimal : we are going through all the nnz values without considering the repartition of it. That is why we provided the CSR (*Compressed Sparse Row* that tends to solve this issue.

B. CSR

In this method, we sort the 3 vectors obtained from the COO format (see II-A) according to the following order for each coordinate (i, j) :

$(i, j) < (i', j') \iff (i < i') \vee (i = i' \wedge j < j')$ where i is the row and j the column.

Then, we replace the *row* array by a *row_ptr* array. This allows us to represent the distribution of non-zero elements by computing a prefix sum of the number of non-zero values per row.

The first element is initialized to 0, and each row $i \in [1, nbRows - 1]$ satisfies: $row_ptr[i] - row_ptr[i - 1]$ for the number of nz in this row i . We provide a sequential solution of CPU where we detail the computation of this *row_ptr*, the 3 vectors are sorted via the *qsort* function provided in C where we copy them in a structure before calling this CSR function. We can write in few lines the operations :

Algorithm 4 SpMV in CSR format (CPU)

```
1: for  $i = 0$  to  $row - 1$  do  
2:   for  $j = row\_ptr[i]$  to  $row\_ptr[i + 1] - 1$  do  
3:      $y[i] \leftarrow y[i] + vals[j] \times x[cols[j]]$   
4:   end for  
5: end for
```

The complexity of this algorithm in a sequential way is in $O(nnz + row)$ for the construction of the *row_ptr* array then for the SpMV it is $O(nnz)$. Finally we can consider that the global complexity is $O(2nnz)$ which shrinks the complexity compared to II-A.

In the GPU solution we provide a naive one using global memory (we call it *grid stride loop format*) that reuses the vectors obtained via the CPU CSR solution and another one using shared memory and warp (see II-C) :

Algorithm 5 SpMV CSR on GPU (grid-stride version)

Require: $y, rows, cols, vals, row_ptr, x, n$

```
1:  $idx \leftarrow threadIdx.x + blockIdx.x \times blockDim.x$   
2: for  $i = idx$  to  $n - 1$  step  $blockDim.x \times gridDim.x$  do  
3:   for  $j = row\_ptr[i]$  to  $row\_ptr[i + 1] - 1$  do  
4:      $y[i] \leftarrow y[i] + vals[j] \times x[cols[j]]$   
5:   end for  
6: end for
```

First loop : Grid-stride loop over rows

Second loop (inner one) : Process all non-zeros of row i

The kernel variable `numBlocks` we use here is defined by $numBlocks = (rows + threadsPerBlock - 1) / threadsPerBlock$ as we are working on lines, so the grid size has to be adapted. But here we can have some issues if we do not use the shared memory and warps. For instance without warp-level parallelism, each row is processed by a single thread. If a row contains many non-zero elements, this thread becomes a bottleneck. That is why we provide a warp/shared memory solution in the next sub section.

C. CSR Warp/Shared memory

To address these limitations, we moved towards a warp-based CSR implementation combined with shared memory usage. The goal is to better balance the workload across threads and reduce execution divergence by distributing the computation of a single row among multiple threads. As we use the shared memory a third parameter is added to the kernel which is computer as follow : $warpsPerBlock = threadsPerBlock / 32$
 $sharedMemSize = threadsPerBlock * sizeof(float)$; We need also to change the `numBlocks` variable due to the fact that we don't go only through lines : $numBlocks = (rows + warpsPerBlock - 1) / warpsPerBlock$. We define by [4] within a thread block, threads are organized into groups of 32 threads called warps. A warp executes the kernel code in a Single-Instruction Multiple-Threads (SIMT) paradigm. In SIMT, all threads in the warp are executing the same kernel code, but each thread may follow different branches through the code. So, threads do not need to follow the same execution path. When threads are executed by a warp, they are assigned a warp lane. Warp lanes are numbered 0 to 31. Here is the solution we provided by storing the partial sum in the shared memory :

For the last part of this pseudo-code (*reduction tree*) the partial results stored in shared memory are combined using this tree scheme, where pairs of values are iteratively summed until a single final result is obtained per warp, each lane processes a subset of the row

We used `__syncthreads()` as a barrier synchronization that forces all threads within a block to wait until every thread has reached the same point of execution. in this case precisely it ensures correct ordering of shared memory writes before performing the tree-based reduction across threads.

D. Others (ELL + SELL

Due to the fact that we the SELL/ELL is not mandatory to implement and we do not have access to the correct version of cuSPARSE library (*see II*) we provide solution on our github that could allow us to compare them with the two previous one. Consequently we can not assume it works correctly for all the matrices of the dataset.

In few lines, SELL SpMV ([3]) to that picture, other terms the user choose the size of a slice that will divide the matrix (in term of row size) and then the width of it will be the maximum number of nnz in the lines. Ellpack is a SELL with a $sliceSize = nbRows$.

Algorithm 6 SpMV CSR Warp GPU implementation with shared memory

Require: $y, rows, cols, vals, row_ptr, x, nnz$
1: Allocate shared memory array *sharedArray*
2: $tid \leftarrow threadIdx.x$
3: $lane \leftarrow tid \bmod 32$
4: $warpInBlock \leftarrow tid / 32$
5: $warp_id \leftarrow blockIdx.x \times (blockDim.x / 32) + warpInBlock$
6: **if** $warp_id \geq rows$ **then**
7: **return**
8: **end if**
9: $row \leftarrow warp_id$
10: $sum \leftarrow 0$
11: **for** $j = row_ptr[row] + lane$ **to** $row_ptr[row + 1] - 1$ **step** 32 **do**
12: $sum \leftarrow sum + vals[j] \times x[cols[j]]$
13: **end for**
14: $sharedArray[warpInBlock \times 32 + lane] \leftarrow sum$
15: **sync threads**
16: **if** $lane < 16$ **then**
17: add neighbor value with offset 16
18: **end if**
19: **sync threads**
20: **if** $lane < 8$ **then**
21: add neighbor value with offset 8
22: **end if**
23: **sync threads**
24: **if** $lane < 4$ **then**
25: add neighbor value with offset 4
26: **end if**
27: **sync threads**
28: **if** $lane < 2$ **then**
29: add neighbor value with offset 2
30: **end if**
31: **sync threads**
32: **if** $lane = 0$ **then**
33: $sharedArray[warpInBlock \times 32] \leftarrow sharedArray[warpInBlock \times 32] + sharedArray[warpInBlock \times 32 + 1]$
34: $y[row] \leftarrow sharedArray[warpInBlock \times 32]$
35: **end if**

III. DATASET

To evaluate our solutions, we use a **dataset of 10 matrices** from **SuiteSparse** [5], a standard reference for sparse matrix benchmarking. These matrices, spanning various applications, allow us to test our implementations under realistic conditions, addressing challenges such as:

- Handling **very large matrices**.
- Optimizing for **irregular structures**.
- Accounting for **symmetry or asymmetry**.

The results obtained from this dataset will enable us to objectively compare the performance of our methods.

	Rows	Cols	Nnz	Kind	Symmetric
cage15	85,623	85,623	2,374,949	Computational Fluid Dynamics Problem	No
ASIC_680ks	682,712	682,712	1,693,767	Circuit Simulation Problem	No
vas_stokes_2M	2,146,677	2,146,677	65,129,037	Semiconductor Process Problem	No
poisson3Db	85,623	85,623	2,374,949	Computational Fluid Dynamics Problem	No
degme	185,501	659,415	8,127,528	Linear Programming Problem	No
1138_bus	1,138	1,138	4,054	Power Network Problem	Yes
shipsec5	179,86	179,86	4,598,604	Structural Problem	Yes
msdoor	415,863	415,863	19,173,163	Structural Problem	Yes
Ge87H76	112,985	112,985	7,892,195	Quantum Chemistry Problem	Yes
bmw3_2	227,362	227,362	11,288,630	Structural Problem	Yes

IV. RESULTS

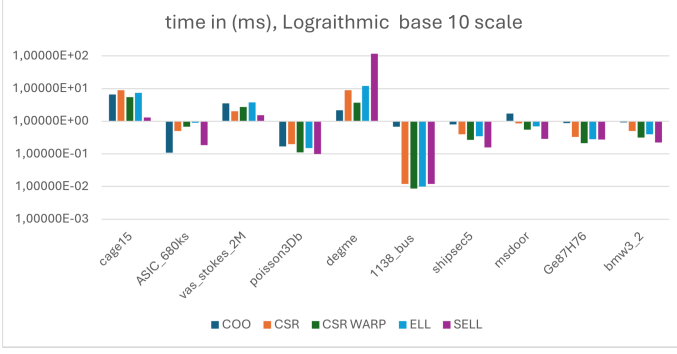


Fig. 1. Mean time comparison between all SpMVs ms (logarithmic scale)

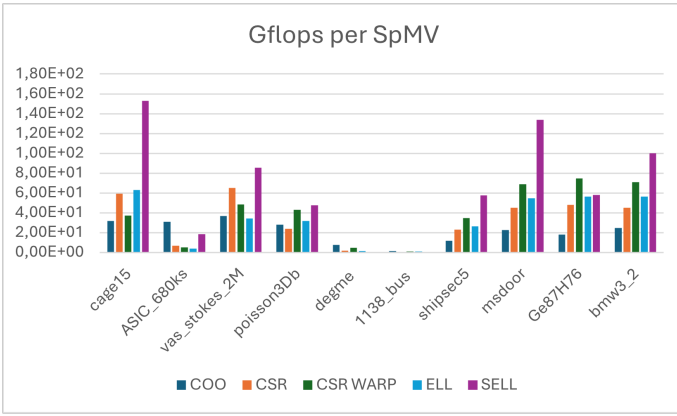


Fig. 2. Mean time comparison between all SpMVs

V. DISCUSSION

in this section we will interpret the results and comment each SpMV methods via [3]

1) COO:

- Performance: Generally consistent but rarely optimal. COO performs well for small or highly irregular matrices (e.g., 1138_bus) but struggles with larger matrices due to poor cache locality.
- Strengths: Simple to implement and parallelize. No pre-processing required.
- Weaknesses: Random memory access patterns lead to poor cache utilization. Redundant storage of row and column indices increases memory bandwidth usage.

2) CSR:

- Performance: Stable baseline but not always the fastest. Outperforms COO for most matrices but struggles with irregular patterns.
- Strengths: Row-wise storage improves cache locality. Standard format widely supported in libraries (e.g., cuSPARSE).
- Weaknesses: Load imbalance in rows with varying non-zero counts can cause warp divergence on GPUs. No explicit vectorization.

- Use Case: Default choice for general-purpose SpMV when no structural information is available.

3) CSR WARP/SHARED MEMORY:

- Performance: Consistently faster than standard CSR,. Achieves 2-5x speedup for matrices like cage15 and ASIC_680ks.
- Strengths: Explicit warp handling minimizes divergence and maximizes memory coalescing. Reduced memory traffic due to warp-level reductions.
- Weaknesses: Overhead for small matrices. Performance gains are hardware-dependent.

4) ELL (ELLpack Format):

- Performance: Mixed results. Excels for regular matrices (e.g., ASIC_680ks) but fails for irregular ones (e.g., degme).
- Strengths: Regular memory access improves cache locality. SIMD-friendly.
- Weaknesses: Padding waste for irregular matrices. Storage overhead for matrices with varying row densities.
- Use Case: Best for matrices with uniform row densities

5) SELL (Sliced Ellpack):

- Performance: Top performer for most matrices (8/10). Achieves the lowest execution times, especially for irregular matrices (e.g., degme).
- Strengths: Hybrid of CSR and ELL. Combines row-wise storage with fixed-length slices to balance locality and flexibility. Reduces padding compared to ELL.
- Weaknesses: Complex tuning required for optimal σ (slice height). Preprocessing overhead.

VI. CONCLUSION

The results demonstrate that no single SpMV format is universally optimal. However, SELL and CSR WARP emerge as the most robust choices for large, irregular matrices on GPUs, while COO and CSR remain viable for simplicity or CPU implementations. The choice of format should be matrix-dependent, with SELL being the safest default for high-performance GPU SpMV (*the state of the art* [1]).

REFERENCES

- [1] "Nvidia Cuspars documentation," docs.nvidia.com/cuda/cuspars/, May 2026.
- [2] "Github of Nvidia Cuspars implementation", github.com/NVIDIA/CUDALibrarySamples/tree/main/cuSPARSE, May 2026.
- [3] Gao, J., Liu, B., Ji, W. and Huang, H., 2024. "A systematic literature survey of sparse matrix-vector multiplication." arXiv preprint arXiv:2404.06047.
- [4] CUDA programming guide, May 2026
- [5] SuiteSparse matrices, May 2026
- [6] G. Eason, B. Noble, and I. N. Sneddon, "On certain integrals of Lipschitz-Hankel type involving products of Bessel functions," Phil. Trans. Roy. Soc. London, vol. A247, pp. 529–551, April 1955.